

## PhonePlan: filosofia della soluzione

I problemi stanno cominciando a farsi strutturalmente complessi ed è quanto mai importante riuscire ad avere una visione complessiva sul sistema prima di iniziare a codificare una soluzione. Per questo è necessario essere certi di avere colto tutti requisiti e anche (soprattutto) i suggerimenti nel testo. Anche in sede d'esame a volte ci sono dei suggerimenti ed è *molto utile* farne tesoro: tipicamente semplificano di molto la soluzione 😊

In questo caso, oltre alla complessità strutturale, i problemi da risolvere sono fondamentalmente due:

1. la determinazione della validità di un piano telefonico
2. il calcolo del costo di una chiamata.

### Validità di un piano telefonico

Le specifiche affermano che tutti gli oggetti coinvolti hanno un metodo `isValid` che consente di chiedere loro se lo stato attuale è consistente o meno.

Per validare un piano occorre verificare che vi siano tariffe (oggetti `Rate`) e che ognuna di queste sia valida.

Per validare una Rate occorre verificare che:

- l'insieme delle `Band` sia non nullo e non vuoto
- ogni `Band` sia valida;
- *l'insieme delle `Band` copra un'intera settimana, senza buchi scoperti.*

Per validare una Band occorre verificare che:

- l'ora di inizio sia precedente all'ora di fine (le ore sono modellate usando `LocalTime` – per i confronti si usano gli appositi metodi);
- l'elenco dei giorni sia non vuoto;
- il costo per intervallo sia non negativo.

Il problema maggiore si incontra quando si deve verificare la copertura della settimana da parte dell'insieme delle `Band` costituenti una tariffa (ossia una `Rate`). Tale oggetto ha tra le sue proprietà l'insieme di fasce orarie (`Band`) in cui applicare quella determinata tariffa.

Serve quindi identificare un algoritmo generale per verificare che l'array di fasce (`private Band[] bands`) offra una copertura 7gg/7 e 24h/24.

**Tariffa TIM scatto alla risposta 15 cent intervalli di 1000 ms applicabile a tutti i numeri che iniziano con "+39339"**

SAB-DOM 10 cent ad intervallo sempre

**LUN-VEN** 08:00-18:30 20 cent ad intervallo

LUN-VEN 18:30-08:00 10 cent ad intervallo

In codice:

```
DayOfWeek[] workWeek = new DayOfWeek[] { DayOfWeek.MONDAY, DayOfWeek.TUESDAY,
                                           DayOfWeek.WEDNESDAY, DayOfWeek.THURSDAY, DayOfWeek.FRIDAY };
DayOfWeek[] weekEnd = new DayOfWeek[] { DayOfWeek.SATURDAY, DayOfWeek.SUNDAY };
Band[] bands = new Band[] {
    new Band(LocalTime.MIN, LocalTime.MAX, weekEnd, 10),
    new Band(LocalTime.of(0, 0), LocalTime.of(8, 0).minusNanos(1), workWeek, 10),
    new Band(LocalTime.of(8, 0), LocalTime.of(18, 30).minusNanos(1), workWeek, 20),
    new Band(LocalTime.of(18, 30), LocalTime.MAX, workWeek, 10) };
Rate r = new Rate("TIM", bands, 60000, 15, "+39339");
PhonePlan plan = new PhonePlan("Piano TIM valido", new Rate[] {r});
assertTrue(plan.isValid());
```

Tariffa TIM scatto alla risposta 15 cent intervalli di 1000 ms applicabile a tutti i numeri che iniziano con "+39339"

SAB-DOM 10 cent ad intervallo sempre

LUN-MER 08:00-18:30 20 cent ad intervallo (manca la copertura GIO-VEN di questa fascia oraria!)

LUN-VEN 18:30-08:00 10 cent ad intervallo

In codice:

```
DayOfWeek[] halfWeek = new DayOfWeek[] { DayOfWeek.MONDAY, DayOfWeek.TUESDAY,
                                           DayOfWeek.WEDNESDAY };
bands = new Band[] { new Band(LocalTime.MIN, LocalTime.MAX, weekEnd, 10),
    new Band(LocalTime.of(0, 0), LocalTime.of(8, 0).minusNanos(1), workWeek, 10),
    new Band(LocalTime.of(8, 0), LocalTime.of(18, 30).minusNanos(1), workWeek, 20),
    new Band(LocalTime.of(18, 30), LocalTime.MAX, halfWeek, 10) };
r = new Rate("TIM", bands, 60000, 15, "+39339");
plan = new PhonePlan("Piano TIM invalido", new Rate[] {r});
assertFalse(plan.isValid());
```

Pensando al processo mentale che seguiremmo “a mano” per validare l’insieme delle bande, l’algoritmo deve verificare la copertura della settimana verificando la copertura di ogni singolo giorno, secondo questo schema:

Per ogni giorno della settimana:

1. Selezionare dall’insieme tutte e sole le Band relative al giorno corrente
2. Ordinare le Band selezionate secondo l’orario di inizio
3. verificare che (si ricordi che l’intervallo di ricerca è chiuso sia a sinistra, sia a destra):
  - i. L’ora di fine di ogni Band selezionata venga un nanosecondo prima dell’ora di inizio della Band successiva (il nanosecondo è la risoluzione di un `LocalTime`)
  - ii. La prima Band inizi alle 00:00:00 + 0 ns (`LocalTime.MIN`)
  - iii. L’ultima Band termini alle 23:59:59 + (10<sup>9</sup> – 1) ns (`LocalTime.MAX`)

Quindi, il metodo `rate.isValid` si occuperà di scorrere i 7 giorni della settimana e verificare che ogni singolo giorno sia coperto totalmente: a questo fine introduciamo il metodo privato ausiliario `validateDay(day)`, così da poterci concentrare su un sotto-problema per volta. In codice:

```

public boolean isValid() {
    // verifica che ogni singola band sia valida (punto a) algoritmo)
    // se anche una sola non è valida la tariffa non è valida
    for (Band band : bands) {
        if (!band.isValid()) {return false;}
    }
    // verifica che ogni giorno sia valido (punto b) algoritmo)
    // se anche uno solo non è valido la tariffa non è valida
    for (DayOfWeek day : DayOfWeek.values()) {
        if (!validateDay(day)) {return false;}
    }
    return true;
}

```

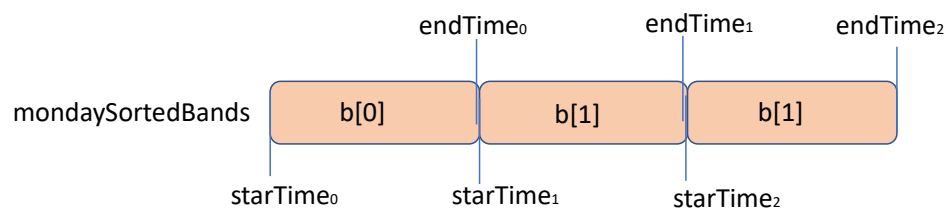
A sua volta, `validateDay(day)` dovrà quindi verificare la copertura del giorno selezionato applicando i punti 1-2-3 dell'algoritmo descritto in precedenza, a cui faremo corrispondere – sia per leggibilità, sia perché stiamo seguendo un approccio metodologico top-down – altrettanti metodi ausiliari e altrettante righe di codice:

```

private boolean validateDay(DayOfWeek day) {
    Band[] bandsInDay = selectBandsInDay(day); //1) seleziona tutte la band di day
    sortBandsByStartTime(bandsInDay); //2) le ordina in base ad orario iniziale
    return validateBandsInDay(bandsInDay); //3) ne verifica l'adiacenza
}

```

Dunque, partendo da lunedì, `selectBandsInDay` (punto 1) ritornerà l'array di bande che hanno quel giorno tra quelli da loro supportati, poi `sortBandsByStartTime` (punto 2) effettuerà l'ordinamento in base all'orario di inizio di ogni banda (classico algoritmo di sort da Fondamenti T1), generando così un array analogo al seguente:



Infine, il punto 3) verificherà l'adiacenza di `b[0]` con `b[1]` (ovvero `endTime0 + 1 nanosecondo = startTime1`), poi di `b[1]` con `b[2]`, e così via. Lo start time della prima banda deve essere uguale a `LocalTime.MIN` (costante che indica l'istante iniziale della giornata) e l'end time dell'ultima banda uguale a `LocalTime.MAX` (costante che indica la fine della giornata). L'utilizzo delle due costanti è utile per evitare di esprimere l'istante di fine senza fare ipotesi sull'ultimo nanosecondo della giornata (`LocalTime.MAX = the maximum supported LocalTime, '23:59:59.999999999'` `LocalTime.MIN = The minimum supported LocalTime, '00:00'`)

```

private boolean validateBandsInDay(Band[] bandsInDay) {
    for (int i = 0; i < bandsInDay.length - 1; i++) {
        LocalTime firstBandEndTime = bandsInDay[i].getEndTime();
        LocalTime secondBandStartTime = bandsInDay[i + 1].getStartTime();

        if (!firstBandEndTime.plusNanos(1).equals(secondBandStartTime)) {return false;}
    }
    return bandsInDay.length > 0
        && bandsInDay[0].getStartTime().equals(LocalTime.MIN)
        && bandsInDay[bandsInDay.length - 1].getEndTime().equals(LocalTime.MAX);
}

```

Si noti come il codice sia perfettamente corrispondente all'algoritmo proposto in linguaggio corrente: questo è cruciale sia per ritrovarci, sia per assicurarsi di non aver aggiunto o dimenticato passi fondamentali, sia, infine, per progettare al meglio il collaudo. Come già osservato sopra, è un classico approccio top-down: il problema viene suddiviso in sotto-problemi di più facile soluzione, che poi si implementano separatamente. In questo modo, prima vengono implementati i metodi di alto livello che delegano (gran) parte del problema ad altri metodi di più basso livello: nel nostro caso, possiamo quindi implementare innanzitutto i metodi `isValid`, poi `validateDay`, poi tutti gli altri. L'obiettivo è avere codice che "si legge" senza la necessità di scrivere particolari commenti; questo non significa che non si debba commentarlo, ma che non è *indispensabile* leggere il commento per capire la logica – il commento dovrebbe servire solo ad *aggiungere dettagli* – poiché il codice scritto è già leggibile e comprensibile. Analogamente si procede per gli altri metodi: `selectBandsInDay` e `sortBandsByStartTime` sono riportati nella soluzione.

### Calcolo del costo di una chiamata

Il calcolo del costo di una chiamata (con le semplificazioni già introdotte nelle slide) dati ora di inizio, ora di fine e numero di destinazione può essere effettuato dall'entità `PhonePlan` mediante l'algoritmo che segue:

1. Selezionare dall'elenco delle tariffe quella applicabile (metodo `isApplicableTo`)
2. Delegare alla tariffa il calcolo del costo

In codice all'interno della classe `PhonePlan`:

```
public double getCallCost(PhoneCall call) {
    Rate selected = getRate(call); //seleziona tra le tariffe quella da applicare
    if (selected == null) {
        return -1; //se non ne trova restituisce -1 come errore
    }
    //altrimenti delega alla tariffa applicabile l'effettivo calcolo del costo
    return selected.getCallCost(call);
}
```

Notare che, anche per un'attività semplice come la ricerca di un oggetto in una collezione, sia stato previsto un opportuno metodo (`getRate`). Questo metodo altro non fa che scorrere l'array di tariffe (proprietà del `PhonePlan`) e selezionare la prima che trova essere applicabile (che sarà la prima tariffa il cui `rootNumber` coincide con il prefisso del numero destinatario, ossia in cui il metodo `r.isApplicableTo(number)` è vero).

```
private Rate getRate(PhoneCall phoneCall) {
    String number = phoneCall.getDestNumber();
    for (Rate r : rates) {
        if (r.isApplicableTo(number)) {
            return r;
        }
    }
    return null;
}
```

Il calcolo effettivo del costo della chiamata spetta alla tariffa, che lo calcolerà attraverso il suo metodo `getCallCost` (metodo `getCallCost` di `Rate`). Questo metodo:

1. In base all'orario di inizio della chiamata trova la Banda da applicare e quindi il costo per intervallo
2. Calcola la durata della chiamata
3. Calcola il numero di intervalli (scatti) da fatturare dividendo la durata della chiamata per la durata di ogni intervallo
4. Moltiplica gli intervalli trovati per il costo trovato al punto 1
5. Aggiunge lo scatto alla risposta e restituisce il valore complessivo del costo

OCCHIO che al punto 3) emerge un problema di divisione tra interi: in Java una tale divisione restituisce sempre un intero, il che implica che se siamo “nel mezzo” di un intervallo il valore decimale venga *troncato*. Questo è sbagliato: l’operatore telefonico non regala nulla, se si deborda da un intervallo occorre pagare tutto l’intervallo successivo. *Ergo, occorre trovare un modo per arrotondare il risultato all’intero superiore*.

Per fare mente locale, prendiamo caso concreto: supponiamo di avere intervalli della durata di 1 minuto e che una chiamata duri 4 minuti e 15 secondi (255000 ms). Il risultato corretto sarebbe 4,25 intervalli, da arrotondarsi a 5 intervalli per il motivo sopra citato, mentre la divisione fra interi restituirebbe 4 determinando una sottostima del costo effettivo, a danno dell’operatore.

Un modo per arrotondare per eccesso è sfruttare la funzione `Math.ceil`, che però va usata con intelligenza: in particolare, NON basta scrivere:

```
Math.ceil(duration/intervalInMillis);
```

perché così facendo il valore passato in ingresso al metodo `ceil` risulterebbe già troncato dalla divisione fra interi, rendendo inutile l’arrotondamento stesso. Per ottenere il risultato desiderato dobbiamo quindi forzare la divisione ad essere tra **double**, con opportuno cast di uno dei due operandi, così che il risultato sia **double**. Nell’esempio sotto si forza la conversione a **double** dell’intero `duration`:

```
int intervalCount = (int)Math.ceil(((double) duration) / intervalInMillis);
```

In alternativa, una seconda soluzione – del tutto diversa come approccio ma che consegue il medesimo risultato – consiste nell’*aggiungere 1 intervallo in caso la divisione tra interi abbia resto maggiore di zero*, una condizione facilmente verificabile con l’operazione di modulo:

```
// Ok divisione intera
int intervalCount = (int) (difference / intervalInMillis);
//ma se sono a metà intervallo arrotondo per eccesso
if((difference % intervalInMillis)!=0)
    intervalCount = intervalCount+1;
```

Quale che sia la soluzione prescelta, il metodo di calcolo costo chiamata della tariffa diventa infine:

```
public double getCallCost(PhoneCall call) {
    // 1) in base all’orario di inizio chiamata trova la banda da applicare e quindi
    // il costo per intervallo
    double costPerInterval = getCostPerInterval(call.getStart());
    if (costPerInterval == -1) {return -1;}

    // 2) calcola la durata della chiamata
    long duration = Duration.between(call.getStart(), call.getEnd()).toMillis();
    // 3) calcola il numero di intervalli
    int intervalCount = (int)Math.ceil(((double) duration) / intervalInMillis);

    // 4 e 5) ritorna il costo della chiamata
    return startCallCost + intervalCount * costPerInterval;
}

private double getCostPerInterval(LocalDateTime dateTime) {
    for (Band b : bands) {
        if (b.isInBand(dateTime)) {
            return b.getCostPerInterval();
        }
    }
    return -1;
}
```

Anche in questo caso, la ricerca della banda è effettuata da un metodo ausiliario.

Il processo di soluzione top-down è ancora più evidente nel caso della valutazione della validità della tariffa in termini di copertura delle bande. Come già mostrato nelle slide, ad ogni passo dell'algoritmo corrisponde un opportuno metodo (privato).

È interessante notare come chi osservi la sola interfaccia pubblica della classe PhonePlan non abbia idea di come sia effettuato dentro il calcolo del costo della chiamata: questo è esattamente ciò che desideriamo, per garantire il massimo disaccoppiamento. Lo si ottiene grazie all'incapsulamento che nasconde i dettagli implementativi.

Roberta Calegari & Ambra Molesini